

HTML SVG

A Practical, From-Zero Learning Guide

Part 6 — Clipping, Masking & Filters

Clipping, masking and filters let SVG go beyond basic shapes. They allow you to crop artwork into custom silhouettes, fade areas gradually, blur objects, create shadows and combine visual effects.

Goal: Understand the difference between clipping and masking, learn the major filter primitives, and build practical visual effects.

1. The Big Picture

Three concepts are important:

- **Clipping** — defines a hard visible boundary.
- **Masking** — controls visibility using a mask, allowing gradual transparency.
- **Filtering** — modifies the rendered appearance using effects such as blur, shadows and color changes.

2. clipPath — The Basic Idea

A **<clipPath>** defines a region in which the target remains visible. Anything outside the clipping region is hidden.

```
<svg viewBox="0 0 300 200">
  <defs>
    <clipPath id="circleClip">
      <circle cx="150" cy="100" r="70"/>
    </clipPath>
  </defs>

  <rect x="20" y="20"
        width="260" height="160"
        fill="tomato"
        clip-path="url(#circleClip)"/>
</svg>
```

The rectangle is still a rectangle, but only the part inside the circle-shaped clipping region is visible.

3. clipPath with Multiple Shapes

```
<clipPath id="windowClip">
  <rect x="40" y="40"
        width="220" height="120"
        rx="20"/>
  <circle cx="150" cy="100" r="45"/>
</clipPath>
```

A clipping path can contain multiple graphical shapes. Their combined geometry determines the clipping region.

4. Clip a Group

```
<defs>
  <clipPath id="clip">
    <circle cx="150" cy="100" r="70"/>
  </clipPath>
</defs>

<g clip-path="url(#clip)">
  <rect width="300" height="200" fill="orange"/>
  <circle cx="100" cy="100" r="60" fill="blue"/>
  <circle cx="200" cy="100" r="60" fill="purple"/>
</g>
```

Clipping a group is useful when many elements must share the same boundary.

5. clipPathUnits

```
<clipPath id="clip"
          clipPathUnits="objectBoundingBox">
  <circle cx="0.5" cy="0.5" r="0.5"/>
</clipPath>
```

objectBoundingBox makes coordinates relative to the target's bounding box. **userSpaceOnUse** uses the SVG's normal coordinate system.

6. Clipping an Image

One of the most common real-world uses is creating circular or custom-shaped images.

```
<defs>
  <clipPath id="avatarClip">
    <circle cx="100" cy="100" r="70"/>
  </clipPath>
</defs>

<image href="photo.jpg"
  x="20" y="20"
  width="160" height="160"
  preserveAspectRatio="xMidYMid slice"
  clip-path="url(#avatarClip)"/>
```

This technique is useful for profile images, thumbnails and cards.

7. clipPath vs border-radius

For simple HTML layouts, CSS **border-radius** is often easier. SVG clipping becomes more useful when the boundary is a custom SVG shape, star, path or combination of shapes.

8. Masking — The Basic Idea

A **<mask>** controls how much of the target is visible. Unlike a simple clip, a mask can create partially transparent regions.

```
<svg viewBox="0 0 300 180">
  <defs>
    <mask id="fadeMask">
      <rect width="300" height="180"
        fill="white"/>
      <circle cx="150" cy="90" r="70"
        fill="black"/>
    </mask>
  </defs>

  <rect width="300" height="180"
    fill="tomato"
    mask="url(#fadeMask)"/>
</svg>
```

In a simple luminance mask, white areas reveal the target while black areas hide it. Gray areas can produce partial visibility.

9. White, Black and Gray in Masks

```
<mask id="mask">
  <rect width="300" height="180" fill="white"/>
  <circle cx="100" cy="90" r="50" fill="gray"/>
  <circle cx="200" cy="90" r="50" fill="black"/>
</mask>
```

Conceptually: white $\approx$ fully visible, black $\approx$ invisible, gray $\approx$ partially visible.

10. Mask with a Gradient

Gradients make masks especially powerful because they can create smooth fades.

```
<defs>
  <linearGradient id="fadeGradient"
    x1="0%" y1="0%"
    x2="100%" y2="0%">
    <stop offset="0%" stop-color="white"/>
  </linearGradient>
</defs>
```

```

    <stop offset="100%" stop-color="black"/>
  </linearGradient>

  <mask id="fadeMask">
    <rect width="300" height="180"
      fill="url(#fadeGradient)"/>
  </mask>
</defs>

<rect width="300" height="180"
  fill="royalblue"
  mask="url(#fadeMask)"/>

```

This creates a smooth transparency transition.

11. clipPath vs mask

Feature	clipPath	mask
Boundary	Hard	Can be soft
Partial transparency	No	Yes
Typical use	Crop/shape	Fade/transparency
Can use gradients	Not as a paint effect	Yes
Complex visual effects	Limited	Powerful

12. The Element

Filters are defined inside **<defs>** and applied with **filter="url(#id)"**.

```

<defs>
  <filter id="myFilter">
    <!-- filter primitives go here -->
  </filter>
</defs>

<circle cx="100" cy="100" r="50"
  filter="url(#myFilter)"/>

```

A filter can contain one or more filter primitives.

13. feGaussianBlur

The most common filter primitive is Gaussian blur.

```

<filter id="blur">
  <feGaussianBlur stdDeviation="8"/>
</filter>

<circle cx="100" cy="100" r="50"
  fill="orange"
  filter="url(#blur)"/>

```

Larger **stdDeviation** values generally create a stronger blur.

14. Blur a Group

```

<filter id="groupBlur">
  <feGaussianBlur stdDeviation="5"/>
</filter>

<g filter="url(#groupBlur)">
  <circle cx="80" cy="100" r="50" fill="red"/>
  <circle cx="140" cy="100" r="50" fill="blue"/>
</g>

```

Filters can be applied to groups as well as individual shapes.

15. Drop Shadows with feDropShadow

```
<filter id="shadow"
  x="-30%" y="-30%"
  width="160%" height="160%">
  <feDropShadow dx="5" dy="8"
    stdDeviation="6"
    flood-opacity="0.35"/>
</filter>

<rect x="50" y="40"
  width="180" height="100"
  rx="20"
  fill="white"
  filter="url(#shadow)"/>
```

The filter region is expanded so the shadow has room to render outside the shape.

16. feOffset

feOffset shifts a filter result.

```
<filter id="offset">
  <feOffset dx="10" dy="10"/>
</filter>
```

By itself it moves the filtered graphic. It becomes more useful when combined with blur and compositing.

17. Building a Shadow Manually

```
<filter id="manualShadow"
  x="-30%" y="-30%"
  width="160%" height="160%">

  <feGaussianBlur in="SourceAlpha"
    stdDeviation="5"
    result="blur"/>

  <feOffset in="blur"
    dx="6" dy="8"
    result="offsetBlur"/>

  <feFlood flood-color="black"
    flood-opacity="0.35"
    result="shadowColor"/>

  <feComposite in="shadowColor"
    in2="offsetBlur"
    operator="in"
    result="shadow"/>

  <feMerge>
    <feMergeNode in="shadow"/>
    <feMergeNode in="SourceGraphic"/>
  </feMerge>
</filter>
```

This introduces the important concept of filter inputs and intermediate results.

18. SourceGraphic and SourceAlpha

- **SourceGraphic** refers to the original rendered graphic.
- **SourceAlpha** refers to the alpha/transparency information of the original graphic.

These special inputs are commonly used when building shadows and compositing effects.

19. Filter Results

Many filter primitives can store their output using **result** and later primitives can use that result.

```
<feGaussianBlur in="SourceGraphic"
  stdDeviation="4"
  result="blurred"/>

<feOffset in="blurred"
  dx="4" dy="4"
  result="moved"/>
```

20. feMerge

feMerge combines multiple filter results.

```
<feMerge>
  <feMergeNode in="shadow"/>
  <feMergeNode in="SourceGraphic"/>
</feMerge>
```

This is a standard technique for putting a shadow behind the original graphic.

21. feFlood

feFlood creates a solid color region that can then be composited into another shape.

```
<feFlood flood-color="#7c3aed"
  flood-opacity="0.5"
  result="color"/>
```

22. feComposite

feComposite combines two inputs according to a compositing operator.

```
<feComposite in="color"
  in2="SourceAlpha"
  operator="in"/>
```

This keeps the flood color only where the source graphic has alpha.

23. Common Composite Operators

- **in** — keeps the first input where the second input exists.
- **out** — keeps the first input outside the second.
- **over** — places one result over another.
- **atop** — combines using the alpha of the second input.
- **xor** — keeps non-overlapping portions.

24. feColorMatrix

A color matrix can modify color channels and create advanced effects such as grayscale, brightness changes and color transformations.

```
<filter id="gray">
  <feColorMatrix type="saturate"
    values="0"/>
</filter>
```

```
<image href="photo.jpg"
      width="300" height="200"
      filter="url(#gray)"/>
```

Using **saturate** with zero saturation creates a grayscale effect.

25. Color Saturation

```
<filter id="desaturate">
  <feColorMatrix type="saturate"
    values="0.25"/>
</filter>
```

Values around 1 preserve normal saturation; lower values reduce saturation.

26. feComponentTransfer

This primitive can modify individual color channels and alpha using transfer functions.

```
<filter id="contrast">
  <feComponentTransfer>
    <feFuncR type="linear"
      slope="1.2"
      intercept="-0.1"/>
    <feFuncG type="linear"
      slope="1.2"
      intercept="-0.1"/>
    <feFuncB type="linear"
      slope="1.2"
      intercept="-0.1"/>
  </feComponentTransfer>
</filter>
```

It is more advanced, but useful when you need precise channel adjustments.

27. feBlend

feBlend blends two inputs using blend modes.

```
<feBlend in="SourceGraphic"
  in2="BackgroundImage"
  mode="multiply"/>
```

Common modes include **normal**, **multiply**, **screen**, **darken** and **lighten**. Support and behavior depend on the rendering context.

28. feTurbulence

feTurbulence generates procedural noise. It can be used for textures, clouds, smoke and distortion effects.

```
<filter id="noise">
  <feTurbulence type="fractalNoise"
    baseFrequency="0.03"
    numOctaves="3"/>
</filter>

<rect width="300" height="200"
  filter="url(#noise)"/>
```

It becomes much more useful when combined with other filter primitives.

29. feDisplacementMap

This primitive uses one image to distort another.

```
<filter id="distort">
```

```

    <feTurbulence type="turbulence"
      baseFrequency="0.02"
      numOctaves="2"
      result="noise"/>

    <feDisplacementMap in="SourceGraphic"
      in2="noise"
      scale="15"/>
  </filter>

```

It can create wavy, liquid, rough or distorted effects.

30. feMorphology

feMorphology can dilate or erode shapes.

```

<filter id="grow">
  <feMorphology operator="dilate"
    radius="3"/>
</filter>

```

dilate expands opaque regions; **erode** shrinks them.

31. Filter Regions

Filters may render beyond the original object's bounds. If the filter region is too small, effects such as shadows or blur can be clipped.

```

<filter id="safeShadow"
  x="-30%" y="-30%"
  width="160%" height="160%">
  <feDropShadow dx="5" dy="8"
    stdDeviation="6"/>
</filter>

```

When creating strong shadows or blur, remember to provide enough filter region.

32. Filter Units

The **filterUnits** attribute controls how the filter region coordinates are interpreted.

```

<filter id="f"
  filterUnits="objectBoundingBox"
  x="-20%" y="-20%"
  width="140%" height="140%">
</filter>

```

As with clipping and gradients, SVG can use bounding-box-relative or user-space coordinates.

33. Combining Clip + Filter

```

<defs>
  <clipPath id="avatar">
    <circle cx="100" cy="100" r="70"/>
  </clipPath>

  <filter id="soft">
    <feGaussianBlur stdDeviation="2"/>
  </filter>
</defs>

<image href="photo.jpg"
  x="20" y="20"
  width="160" height="160"
  clip-path="url(#avatar)"
  filter="url(#soft)"/>

```


SVG features can be combined. Here an image is clipped to a circle and then blurred.

34. Mini Project — Circular Avatar

```
<svg viewBox="0 0 260 260">
  <defs>
    <clipPath id="avatarClip">
      <circle cx="130" cy="130" r="90"/>
    </clipPath>

    <filter id="avatarShadow"
      x="-30%" y="-30%"
      width="160%" height="160%">
      <feDropShadow dx="0" dy="8"
        stdDeviation="8"
        flood-opacity="0.3"/>
    </filter>
  </defs>

  <circle cx="130" cy="130" r="90"
    fill="#e2e8f0"
    filter="url(#avatarShadow)"/>

  <image href="photo.jpg"
    x="40" y="40"
    width="180" height="180"
    preserveAspectRatio="xMidYMid slice"
    clip-path="url(#avatarClip)"/>
</svg>
```

35. Mini Project — Glow Effect

```
<svg viewBox="0 0 300 200">
  <defs>
    <filter id="glow"
      x="-50%" y="-50%"
      width="200%" height="200%">

      <feGaussianBlur stdDeviation="12"
        result="blur"/>

      <feMerge>
        <feMergeNode in="blur"/>
        <feMergeNode in="SourceGraphic"/>
      </feMerge>

    </filter>
  </defs>

  <circle cx="150" cy="100"
    r="45"
    fill="#38bdf8"
    filter="url(#glow)"/>
</svg>
```

This is a common foundation for neon and glowing UI effects.

36. Mini Project — Fade an Image

```
<svg viewBox="0 0 500 220">
  <defs>
    <linearGradient id="fade"
      x1="0%" y1="0%"
      x2="100%" y2="0%">
      <stop offset="0%" stop-color="white"/>
      <stop offset="100%" stop-color="black"/>
    </linearGradient>

    <mask id="imageFade">
      <rect width="500" height="220"

```

```

        fill="url(#fade)"/>
    </mask>
</defs>

<image href="photo.jpg"
width="500" height="220"
preserveAspectRatio="xMidYMid slice"
mask="url(#imageFade)"/>
</svg>

```

This technique can create elegant hero-image fades.

37. Common Mistakes

- Using clipPath when you actually need partial transparency.
- Forgetting to define clip paths, masks or filters inside defs.
- Using the wrong coordinate system for a clipping or filter region.
- Making a filter region too small and accidentally clipping shadows.
- Using very expensive filters on huge graphics without considering performance.
- Forgetting to give referenced resources unique IDs.
- Creating a complicated filter when a simple CSS effect would be enough.

38. Performance Tips

- Prefer simple shapes and CSS effects when they produce the same result.
- Avoid very large blur radii unless necessary.
- Avoid applying expensive filters to enormous groups.
- Reuse definitions instead of duplicating complex filter definitions.
- Test filter-heavy SVGs on lower-powered devices.

39. Exercises

- Clip a rectangle into a star.
- Create a circular profile image using clipPath.
- Create a diagonal image fade using a mask.
- Make a soft glowing circle with Gaussian blur.
- Create a card with a drop shadow.
- Create a grayscale image using feColorMatrix.
- Experiment with turbulence to create a noise texture.
- Combine turbulence and displacement to create a liquid effect.
- Combine clipping, masking and filtering into one graphic.

40. Quick Reference

Element / Property	Purpose
--------------------	---------

<clipPath>	Defines a hard clipping region
clip-path	Applies a clipping path
clipPathUnits	Controls clipping coordinates
<mask>	Controls visibility/transparency
mask	Applies a mask
<filter>	Defines a visual filter
filter	Applies a filter
<feGaussianBlur>	Blur effect
<feDropShadow>	Drop shadow
<feOffset>	Shift filter content
<feFlood>	Create a solid color input
<feComposite>	Combine/filter two inputs
<feMerge>	Combine filter results
<feColorMatrix>	Transform colors
<feBlend>	Blend filter inputs
<feTurbulence>	Generate procedural noise
<feDisplacementMap>	Distort one input using another
<feMorphology>	Dilate or erode shapes
SourceGraphic	Original graphic input
SourceAlpha	Alpha of original graphic

41. What's Next?

Part 7 will cover **SVG Reuse, Symbols, Markers & Advanced Structure**: <use>, <symbol>, <defs>, reusable components, markers, arrowheads, sprites and building a reusable SVG icon system.